



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES
LICENCIATURA EN CIBERSEGURIDAD

Investigación #1

Asignatura:

Programación 2

Integrante:

Magdalena Gonzalez

Cédula: 4-819-1590

Profesor:

Napoleón Ibarra

Semestre:

Primer Semestre

2026

Introducción

Python ocupa un lugar privilegiado dentro del desarrollo de software actual gracias a su sintaxis clara, su amplia comunidad y su integración con múltiples tecnologías modernas. Comprender su evolución, sus características técnicas y sus estructuras internas no solo permite utilizarlo con mayor eficacia, sino también analizar por qué ha logrado consolidarse en áreas tan diversas como la inteligencia artificial, el desarrollo web y la automatización de sistemas. A lo largo de este trabajo se examina su origen, su relevancia práctica y una comparación detallada de sus principales estructuras de datos.

Historia, fundamentos y estructuras clave del lenguaje de programación Python

Investigación 1

✓ ¿Cuál es su concepto, características?

Python fue creado por Guido van Rossum a finales de la década de 1980 y lanzado oficialmente en 1991. Desde sus primeras versiones, el objetivo principal consistió en ofrecer un lenguaje que combinara simplicidad y potencia, inspirado en lenguajes como ABC, pero con mayor flexibilidad y capacidad de extensión.



Python es un lenguaje de programación interpretado, de alto nivel y multiparadigma. Esto significa que permite trabajar con distintos enfoques, como programación orientada a objetos, programación funcional y programación estructurada.

Entre sus características más relevantes destacan:

a. Sintaxis sencilla y legible

Utiliza una estructura clara basada en indentación, lo que reduce errores comunes y facilita el aprendizaje.

b. Tipado dinámico

No exige declarar explícitamente el tipo de las variables, lo que acelera el desarrollo.

c. Multiplataforma

Puede ejecutarse en diversos sistemas operativos sin modificaciones importantes.

d. Amplia biblioteca estándar

Incluye módulos integrados para manejo de archivos, redes, matemáticas, entre otros.

e. Gran ecosistema de frameworks

Aquí radica uno de sus mayores puntos fuertes: Python se integra con herramientas como Django, Flask, FastAPI, TensorFlow y PyTorch.

✓ *Importancia y relevancia en la actualidad*

Python ha alcanzado una posición dominante por razones que trascienden su facilidad de uso. Su impacto se entiende mejor al analizar tres dimensiones clave:

- **Desarrollo web:** frameworks como Django y Flask permiten crear aplicaciones robustas en menos tiempo.
- **Ciencia de datos e inteligencia artificial:** bibliotecas como NumPy, Pandas y TensorFlow han convertido a Python en el estándar de facto en estas áreas.
- **Automatización y ciberseguridad:** su simplicidad facilita la creación de scripts para análisis de redes, escaneo de vulnerabilidades y pruebas de penetración.

A diferencia de otros lenguajes, Python no solo destaca por su sintaxis, sino por la cohesión de su ecosistema. Los frameworks están diseñados para integrarse de forma natural, lo que reduce la complejidad en proyectos grandes.

Investigación 2

✓ *Comparación: Listas vs Tuplas en Python*

Característica	Listas	Tuplas
Definición	Colección ordenada y mutable	Colección ordenada e inmutable

Sintaxis	[]	()
Modificabilidad	Permite cambios	No permite cambios
Rendimiento	Más flexibles, ligeramente más lentas	Más rápidas por ser inmutables
Uso común	Datos dinámicos	Datos constantes

Las listas ofrecen mayor flexibilidad, lo que resulta útil cuando los datos cambian constantemente. En cambio, las tuplas aportan estabilidad y mejor rendimiento en situaciones donde los valores no deben modificarse. Por ejemplo, en configuraciones o datos estructurados que no cambian durante la ejecución.

✓ **Comparación: Conjuntos vs Diccionarios en Python**

Característica	Conjuntos (set)	Diccionarios (dict)
Concepto	Colección no ordenada sin duplicados	Colección de pares clave-valor
Sintaxis	{1, 2, 3}	{"clave": "valor"}
Acceso a datos	No indexado	Acceso mediante claves
Aplicabilidad	Eliminación de duplicados, operaciones matemáticas	Almacenamiento estructurado de datos
Eficiencia	Alta en operaciones de pertenencia	Alta en búsquedas por clave

Los conjuntos resultan útiles para operaciones como intersecciones o eliminación de duplicados, mientras que los diccionarios permiten representar relaciones más

complejas entre datos. En aplicaciones reales, los diccionarios dominan debido a su capacidad para modelar estructuras tipo base de datos.

✓ **Métodos para la manipulación de listas**

Las listas constituyen una de las estructuras más utilizadas en Python. Su versatilidad se refleja en la variedad de métodos disponibles:

- **append()**: agrega un elemento al final.
- **insert()**: inserta un elemento en una posición específica.
- **remove()**: elimina un elemento por valor.
- **pop()**: elimina y devuelve un elemento según su índice.
- **sort()**: ordena la lista.
- **reverse()**: invierte el orden de los elementos.
- **extend()**: combina listas.

Ejemplo conceptual:

- `append` permite construir listas progresivamente.
- `sort` facilita el procesamiento de datos ordenados.
- `pop` resulta útil en estructuras tipo pila.

Estos métodos evidencian la orientación práctica del lenguaje: operaciones comunes se ejecutan con instrucciones simples y directas.

Conclusión

Python ha evolucionado desde un proyecto personal hasta convertirse en una herramienta esencial en múltiples áreas tecnológicas. Su éxito no depende únicamente de su facilidad de uso, sino de la coherencia entre su diseño, sus estructuras de datos y su ecosistema de frameworks.

El análisis de listas, tuplas, conjuntos y diccionarios demuestra que cada estructura cumple una función específica, lo que permite seleccionar la más adecuada según el problema. A su vez, la integración con frameworks consolida su utilidad en escenarios reales, desde aplicaciones web hasta sistemas de inteligencia artificial.

En conjunto, Python ofrece un equilibrio entre simplicidad y capacidad técnica que lo posiciona como uno de los lenguajes más influyentes en la actualidad.

Referencias

Van Rossum, G., & Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace.

Python Software Foundation. (2024). *Python Documentation*.
<https://docs.python.org/3/>

Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.

Sweigart, A. (2019). *Automate the Boring Stuff with Python* (2nd ed.). No Starch Press.

Real Python. (2024). *Python Tutorials*. <https://realpython.com/>

W3Schools. (2024). *Python Tutorial*. <https://www.w3schools.com/python/>